



Arduino and
Robotics
Pixels



Soccer Car

PROJECT TEAM: **CYBER BEASTS**





Pixels Arduino
and Robotics





***ROBOT
COMPONENTS***



***COMPONENT
SELECTION***



***COST
ANALYSIS***



***MECHANICAL
DESIGN***



Code





Pixels Arduino
and Robotics



ROBOT COMPONENTS

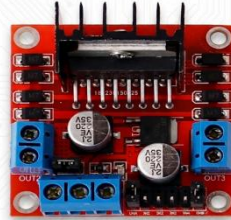
Main components used in our robot

1 MAIN CONTROLLER



- ESP32 Main Board

2 MOTOR DRIVE / H-BRIDGE



- L298N motor drive x1

3 DC GEAR MOTOR



- DC gear motor yellow x 4

4 WHEELS / TIRES



- Rubber Wheels x4

5 18650 BATTERY



- 3 x 18650 li-ion cells

6 BATTERY HOLDER



- 3S battery holder
- Holds 3x 18650 Cells

7 WIRING



- Jumper Wires

8 ROBOT CONTROL

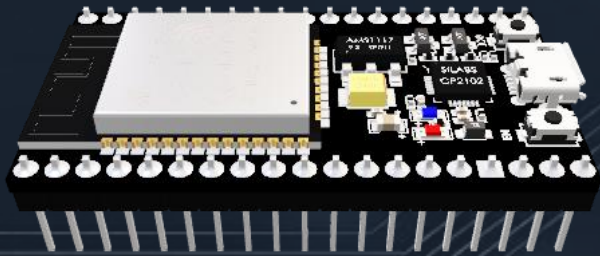


- Smartphone app controller
- 4-way direction control



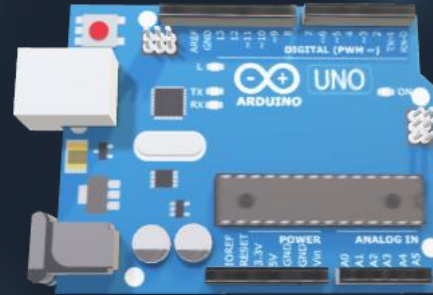
✓ *COMPONENT SELECTION*

Why we choose this components and not others?



Built-in Bluetooth & Wi-Fi: Features integrated Wi-Fi and Bluetooth without needing extra components, at a price of 325 EGP.

Better Performance & Lower Cost: Offers faster processing and larger memory for wireless signals at a lower total price.



Standalone Board Price: The Arduino Uno board alone costs 350 EGP.

Requires External Module: To connect with Application Control or a PlayStation Controller, an external Bluetooth module (e.g., HC-05) is required, costing 200 to 250 EGP.

Total Setup Cost: Total cost with the module ranges between 550 to 600 EGP.



✓ *COMPONENT SELECTION*

Why we choose this components and not others?



Zero Extra Hardware Cost: Free to implement using existing smartphones without purchasing additional hardware.

Custom & Interactive UI: Highly customizable dashboard displaying control buttons, real-time battery status, speed, and sensor feedback.

High Accessibility: Convenient, always available, and easily updated via software without hardware limitations.



High Additional Cost: Expensive hardware ranging from 800 to 1,500+ EGP, significantly increasing project budget.

Fixed Control Layout: Fixed button configuration with no dynamic screen feedback (cannot display live telemetry or speed).

Maintenance & Battery: Requires separate charging and setup maintenance compared to a unified smartphone interface.



✓ *COMPONENT SELECTION*

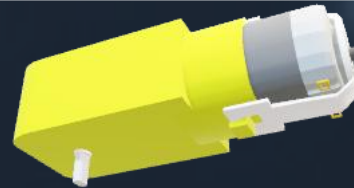
Why we choose this components and not others?



Li-ion vs. LiPo (Durability & Safety): *Li-ion 18650 cells feature a rigid metal casing, making them safer and more shock-resistant than delicate LiPo batteries, which can swell easily under impact.*

Li-ion vs. NiMH (Weight & Output): *Li-ion offers a much higher energy density and voltage per cell compared to NiMH, requiring fewer cells and less weight to reach 11.1V.*

Cost & Charging Efficiency: *18650 Li-ion is the most cost-effective and readily available option, working smoothly with standard BMS circuits.*



Vs. Metal Gear Motor (JGA25):

*The yellow TT motor with its wheel costs only **40 EGP**, whereas the JGA25 costs **250 EGP** per motor (~1000 EGP for four), which is unnecessary and too expensive for this car.*

The metal motor is too heavy, while the yellow TT motor is lightweight and fits the chassis perfectly.

Why 4 Motors vs. (2 Motors + Servo)?

Servos and mechanical steering linkages are more complex and annoying to handle in code.

*Four motors provide **higher speed and stronger torque** for better movement.*



COMPONENT SELECTION

Why we choose this components and not others?

COMPONENT	WHY WE CHOSE IT	WHY NOT OTHERS
 ESP32 Main Board	- Built-in Wi-Fi & Bluetooth for native wireless control. - High processing power & dual-core performance. - Cost-effective & widely supported platform.	Arduino Uno (Wired): Lacks native wireless; adding a Bluetooth module is required, which significantly increases total cost. Raspberry Pi: Excessively high cost & power consumption.
 L298N Motor Driver	- Can drive high current motors (up to 2A per channel). - Reliable, robust, and cost-effective module. - Simple interface for easy connection with ESP32.	L293D: Limited current capacity; cannot handle motor loads. BTS7960: Overkill and unnecessary expense for our current requirements.
 Yellow DC Gear Motors	- Very affordable and universally available. - High torque and good speed for obstacle climbing. - Simple speed & direction control.	Stepper Motors: Require complex control algorithms and high power. Servo Motors: Limited rotation and higher complexity.
 Rubber Wheels	- Perfect fit with yellow DC gear motors. - High traction rubber tires for smooth, non-slip movement.	Plastic Wheels: prone to slippage on smooth surfaces. Omni Wheels: Introduces unnecessary complexity and cost.
 3 x 18650 Li-ion Cells	- Extremely high energy density & power delivery. - Rechargeable and offers long operating time. - Reliable and readily available cells.	AA Alkaline: Non-rechargeable and very low power for motors. NiMH: Much heavier with significantly lower energy density.
 3S Battery Holder	- Provides secure, stable mounting for all three batteries. - Includes built-in leads for convenient and easy wiring.	Direct Soldering: Permanent and creates unsafe connections. Loose Tape/Straps: Unreliable and dangerous cell positioning.
 Jumper Wires	- Flexible and allows for very fast, easy prototyping. - Color-coded organization prevents connection errors.	Permanent Soldering: Time-consuming and very hard to modify. Thick Industrial Cables: excessively bulky for this small robot chassis.
 Robot Control (Smartphone App)	- Wireless 4-way direction & speed control using smartphone. - Utilizes existing user smartphone interface for control. - Entirely zero additional hardware cost.	Physical Remote Controller: Requires purchase of extra, costly hardware. IR Control: Requires direct line-of-sight and is very limited. PlayStation/Dedicated Controller: significantly higher cost to acquire and integrate.





Cost Analysis

	Component	Qty	Status	Unit Price (EGP)	Total Project Value (EGP)	Out-of-Pocket Cost (EGP)
1	 ESP32	1	Existing Stock	325	325	0
2	 L298N Motor Driver	2	Procured	70	140	140
3	 Yellow DC Gear Motors	4	Procured	40	160	160
4	 Rubber Wheels	4	Procured			
5	 3 x 18650 Li-ion Cells	3	Procured	80	240	240
6	 3S Battery Holder	1	Procured	14	14	14
7	 Jumper Wires		Existing Stock	10	10	0
8	 Robot Control (Smartphone App)	0		0	0	0
9	 3D Design & Printing	1	Procured	700	700	700
Grand Total				1239	1589	1254

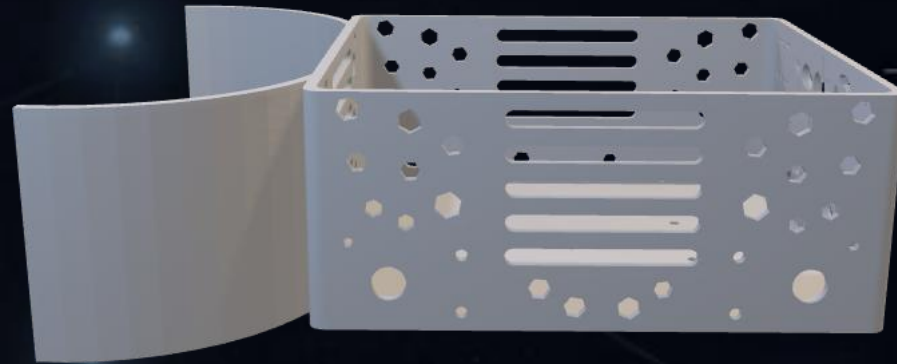




Pixels Arduino
and Robotics



MECHANICAL DESIGN

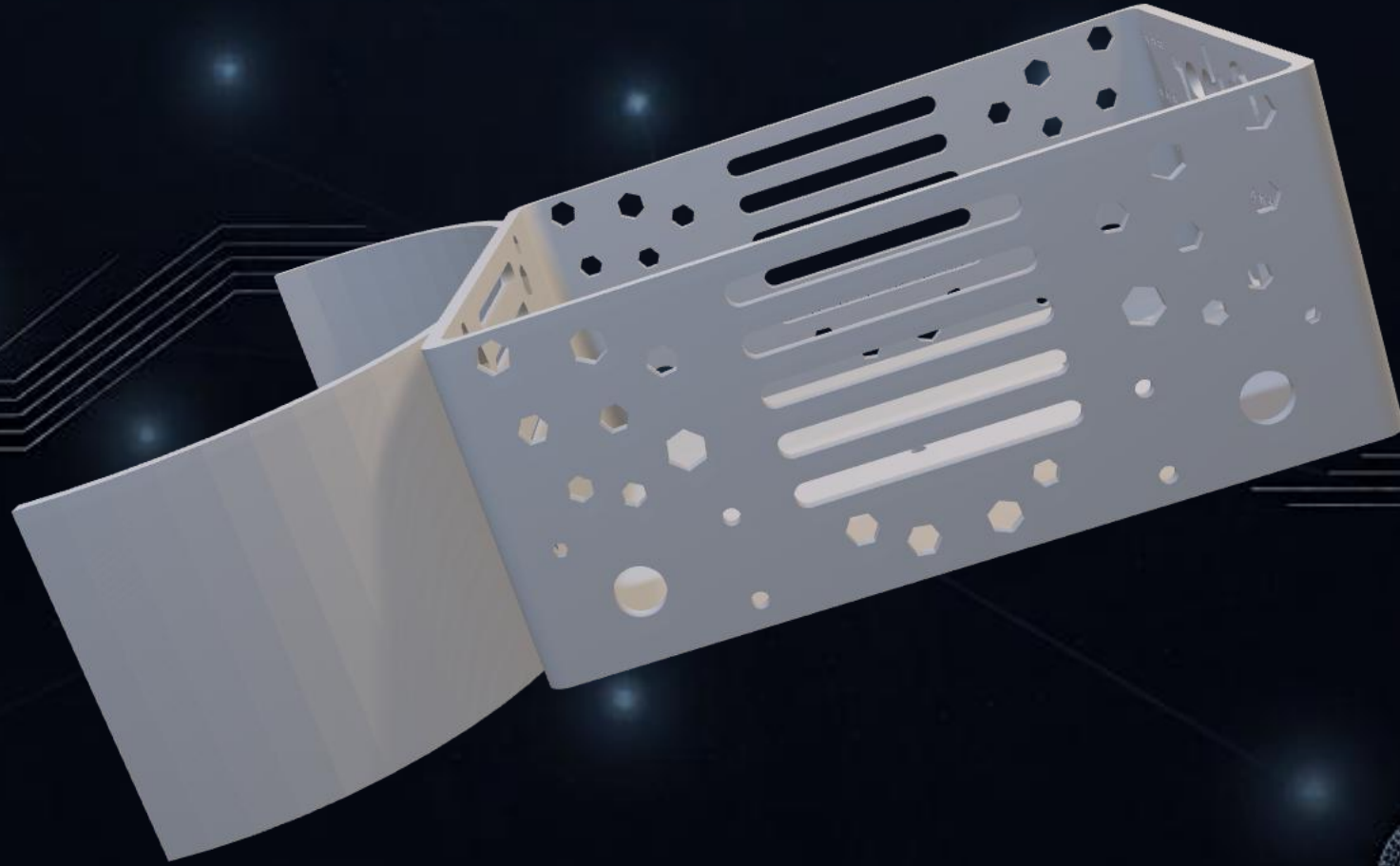




Pixels Arduino
and Robotics



MECHANICAL DESIGN





MECHANICAL DESIGN

Design Purpose:

Ball Control: The curved front bumper secures the ball for smooth dribbling and precise steering.

Component Protection: Box enclosure protects electronics (ESP32, L298N) and motors from external impacts.

Dimensions & Weight:

Length: ~ 24.7 cm (including front curved gripper).

Width: ~ 19.5 cm (including wheels).

Height: ~ 9.2 cm.

Weight: Ultra-lightweight = 538.5 grams (reduces motor load and saves battery power).





Pixels Arduino
and Robotics

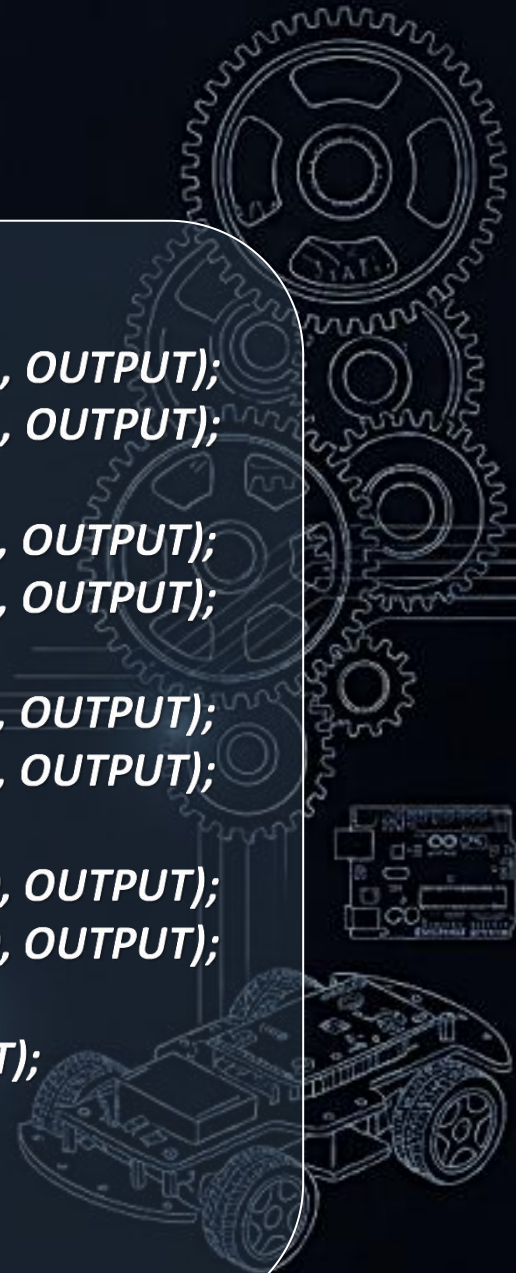


Code

```
#include  
"BluetoothSerial.h"  
  
BluetoothSerial SerialBT;  
  
#define speedMottorA 13  
#define speedMottorB 12  
#define speedMottorC 14  
#define speedMottorD 27  
  
#define IN1MottorA 26  
#define IN2MottorA 25  
  
#define IN1MottorB 33  
#define IN2MottorB 32  
  
#define IN1MottorC 23  
#define IN2MottorC 22
```

```
#define IN1MottorD 19  
#define IN2MottorD 18  
  
#define led 2  
  
unsigned long lastPrint = 0;  
char command = 0;  
  
void setup() {  
  Serial.begin(115200);  
  SerialBT.begin("Syber Beasts");  
  
  pinMode(speedMottorA, OUTPUT);  
  pinMode(speedMottorB, OUTPUT);  
  pinMode(speedMottorC, OUTPUT);  
  pinMode(speedMottorD, OUTPUT);
```

```
  pinMode(IN1MottorA, OUTPUT);  
  pinMode(IN2MottorA, OUTPUT);  
  
  pinMode(IN1MottorB, OUTPUT);  
  pinMode(IN2MottorB, OUTPUT);  
  
  pinMode(IN1MottorC, OUTPUT);  
  pinMode(IN2MottorC, OUTPUT);  
  
  pinMode(IN1MottorD, OUTPUT);  
  pinMode(IN2MottorD, OUTPUT);  
  
  pinMode(led, OUTPUT);
```





Pixels Arduino
and Robotics



Code

```
analogWrite(speedMottorA, 0);  
analogWrite(speedMottorB, 0);  
analogWrite(speedMottorC, 0);  
analogWrite(speedMottorD, 0);  
  
digitalWrite(led, LOW);  
  
Serial.println("Bluetooth  
Started");  
Serial.println("Name:  
ESP32_CAR");  
}
```

```
void loop() {  
  
    while (SerialBT.available()) {  
        char x = SerialBT.read();  
  
        if (x == 'F' || x == 'B' || x == 'L' || x  
            == 'R' ||  
            x == 'Q' || x == 'Z' || x == 'E' || x  
            == 'C' || x == 'S') {  
            command = x;  
        }  
    }  
  
    if (millis() - lastPrint >= 50) {  
        lastPrint = millis();  
  
        switch (command) {
```

```
        case 'F':  
            analogWrite(speedMottorA, 255);  
            digitalWrite(IN1MottorA, HIGH);  
            digitalWrite(IN2MottorA, LOW);  
  
            analogWrite(speedMottorB, 255);  
            digitalWrite(IN1MottorB, HIGH);  
            digitalWrite(IN2MottorB, LOW);  
  
            analogWrite(speedMottorC, 255);  
            digitalWrite(IN1MottorC, HIGH);  
            digitalWrite(IN2MottorC, LOW);  
  
            analogWrite(speedMottorD, 255);  
            digitalWrite(IN1MottorD, HIGH);  
            digitalWrite(IN2MottorD, LOW);  
  
            digitalWrite(led, HIGH);  
            break;
```



Pixels Arduino
and Robotics



Code

```
case 'B':  
analogWrite(speedMottorA, 255);  
digitalWrite(IN1MottorA, LOW);  
digitalWrite(IN2MottorA, HIGH);
```

```
analogWrite(speedMottorB, 255);  
digitalWrite(IN1MottorB, LOW);  
digitalWrite(IN2MottorB, HIGH);
```

```
analogWrite(speedMottorC, 255);  
digitalWrite(IN1MottorC, LOW);  
digitalWrite(IN2MottorC, HIGH);
```

```
analogWrite(speedMottorD, 255);  
digitalWrite(IN1MottorD, LOW);  
digitalWrite(IN2MottorD, HIGH);
```

```
digitalWrite(led, HIGH);  
break;
```

```
case 'L':  
analogWrite(speedMottorA, 255);  
digitalWrite(IN1MottorA, HIGH);  
digitalWrite(IN2MottorA, LOW);
```

```
analogWrite(speedMottorB, 0);  
digitalWrite(IN1MottorB, LOW);  
digitalWrite(IN2MottorB, LOW);
```

```
analogWrite(speedMottorC, 255);  
digitalWrite(IN1MottorC, HIGH);  
digitalWrite(IN2MottorC, LOW);
```

```
analogWrite(speedMottorD, 0);  
digitalWrite(IN1MottorD, LOW);  
digitalWrite(IN2MottorD, LOW);
```

```
digitalWrite(led, HIGH);  
break;
```

```
case 'R':  
analogWrite(speedMottorA, 0);  
digitalWrite(IN1MottorA, LOW);  
digitalWrite(IN2MottorA, LOW);
```

```
analogWrite(speedMottorB, 255);  
digitalWrite(IN1MottorB, HIGH);  
digitalWrite(IN2MottorB, LOW);
```

```
analogWrite(speedMottorC, 0);  
digitalWrite(IN1MottorC, LOW);  
digitalWrite(IN2MottorC, LOW);
```

```
analogWrite(speedMottorD, 255);  
digitalWrite(IN1MottorD, HIGH);  
digitalWrite(IN2MottorD, LOW);
```

```
digitalWrite(led, HIGH);  
break;
```



Pixels Arduino
and Robotics



Code

case 'Q':

```
analogWrite(speedMottorA, 255);  
digitalWrite(IN1MottorA, HIGH);  
digitalWrite(IN2MottorA, LOW);
```

```
analogWrite(speedMottorB, 127);  
digitalWrite(IN1MottorB, HIGH);  
digitalWrite(IN2MottorB, LOW);
```

```
analogWrite(speedMottorC, 255);  
digitalWrite(IN1MottorC, HIGH);  
digitalWrite(IN2MottorC, LOW);
```

```
analogWrite(speedMottorD, 127);  
digitalWrite(IN1MottorD, HIGH);  
digitalWrite(IN2MottorD, LOW);
```

```
digitalWrite(led, HIGH);  
break;
```

case 'Z':

```
analogWrite(speedMottorA, 255);  
digitalWrite(IN1MottorA, LOW);  
digitalWrite(IN2MottorA, HIGH);
```

```
analogWrite(speedMottorB, 127);  
digitalWrite(IN1MottorB, LOW);  
digitalWrite(IN2MottorB, HIGH);
```

```
analogWrite(speedMottorC, 255);  
digitalWrite(IN1MottorC, LOW);  
digitalWrite(IN2MottorC, HIGH);
```

```
analogWrite(speedMottorD, 127);  
digitalWrite(IN1MottorD, LOW);  
digitalWrite(IN2MottorD, HIGH);
```

```
digitalWrite(led, HIGH);  
break;
```

case 'E':

```
analogWrite(speedMottorA, 127);  
digitalWrite(IN1MottorA, HIGH);  
digitalWrite(IN2MottorA, LOW);
```

```
analogWrite(speedMottorB, 255);  
digitalWrite(IN1MottorB, HIGH);  
digitalWrite(IN2MottorB, LOW);
```

```
analogWrite(speedMottorC, 127);  
digitalWrite(IN1MottorC, HIGH);  
digitalWrite(IN2MottorC, LOW);
```

```
analogWrite(speedMottorD, 255);  
digitalWrite(IN1MottorD, HIGH);  
digitalWrite(IN2MottorD, LOW);
```

```
digitalWrite(led, HIGH);  
break;
```



Pixels Arduino
and Robotics



Code

```
case 'C':  
  analogWrite(speedMottorA, 127);  
  digitalWrite(IN1MottorA, LOW);  
  digitalWrite(IN2MottorA, HIGH);  
  
  analogWrite(speedMottorB, 255);  
  digitalWrite(IN1MottorB, LOW);  
  digitalWrite(IN2MottorB, HIGH);  
  
  analogWrite(speedMottorC, 127);  
  digitalWrite(IN1MottorC, LOW);  
  digitalWrite(IN2MottorC, HIGH);  
  
  analogWrite(speedMottorD, 255);  
  digitalWrite(IN1MottorD, LOW);  
  digitalWrite(IN2MottorD, HIGH);  
  
  digitalWrite(led, HIGH);  
  break;
```

```
case 'S':  
  analogWrite(speedMottorA, 0);  
  digitalWrite(IN1MottorA, LOW);  
  digitalWrite(IN2MottorA, LOW);  
  
  analogWrite(speedMottorB, 0);  
  digitalWrite(IN1MottorB, LOW);  
  digitalWrite(IN2MottorB, LOW);  
  
  analogWrite(speedMottorC, 0);  
  digitalWrite(IN1MottorC, LOW);  
  digitalWrite(IN2MottorC, LOW);  
  
  analogWrite(speedMottorD, 0);  
  digitalWrite(IN1MottorD, LOW);  
  digitalWrite(IN2MottorD, LOW);  
  
  digitalWrite(led, LOW);  
  break;  
}  
}  
}
```



CYBER BEASTS

Final project Pixels

Cyber beasts

Final project Pixels

Team
members :

Cyber beasts

Final project Pixels

Team
members :

Sama Mohamed Sayed

Jana Khaled Ebrahim

Shahd Hany Farouk

Ebrahim Elsayed Abdelhamed

Cyber beasts

Final project Pixels

Team
members :

Sama Mohamed Sayed

Jana Khaled Ebrahim

Shahd Hany Farouk

Ebrahim Elsayed Abdelhamed

Cyber beasts

Final project Pixels

Cyber beasts

Final project Pixels

THANK YOU